ENGINEERING DOCUMENT

How we built it

An implementation walkthrough of the Platform Walkthrough Agent: the modules, the wire protocol, the control flow of a single run, and the reasoning behind each piece. Written for whoever has to change this code next.

Backend	Python 3.14 · FastAPI · Playwright 1.62 · Anthropic SDK · 2,150 lines
Frontend	Next.js 15 · React 19 · TypeScript · 1,320 lines
Model	claude-opus-5 — structured outputs for planning and Q&A, prompt caching for the document
Browser	Chrome over the DevTools Protocol, attached rather than launched
Tests	25 hermetic (2.7 s) + 1 real-browser integration test
Deploy	Harness Agent Skill + pipeline, with the test suite as the gate

HOW TO READ THIS

Sections 1-3 are the shape of the system: the run lifecycle, the module layout, and the wire protocol. Sections 4-8 go one level down into the four parts that carry the weight — the browser layer, the planner, the executor, and the overlay. Section 9 onwards covers configuration, testing, and where to extend it.

1 · The lifecycle of one run

Everything else in this document is a detail of this sequence. The ordering is load-bearing: planning happens *after* the agent can see the page, which is what makes the same code work on a product it has never been pointed at before.

```
  presenter clicks Start
        |
        v
  1. resolve the request      DEMO_LOCKED ? the preset : what the client sent
  2. resolve the document     upload id -> disk, or the bundled sample
  3. ensure a browser         attach over CDP; auto-launch Chrome if the
                              debug port is closed  (~2.7s cold)
  4. select the tab           score open tabs against the hint, or refuse
  5. derive the scope         host + registrable domain of that tab
  6. offer the sign-in gate   ask, wait, then verify by reading the page
  7. read the page            OUTLINE_JS -> roles + accessible names
  8. plan                     doc + outline + scope -> StepPlan  (structured)
  9. narrate                  one call fills every step's spoken line
 10. execute                  the pause-gated loop, streaming events
        |
        +-- question? -> freeze, answer, maybe re-plan, resume
        +-- step done? -> ask the room, hold, continue
        v
 11. closing question window, then complete
```

Why steps 3-7 are in that order

A planner that runs before the browser is attached has to invent the page. Every version of this that shipped a hardcoded site did exactly that, and broke the first time the product's markup moved. Attaching first, then reading, then planning means the plan can only reference controls that were on screen a second ago.

The sign-in gate sits at 6 rather than later for the same reason: signed-out and signed-in are different products, and a plan written for one is wrong for the other. Asking after planning would mean throwing the plan away.

2 · Module layout

Backend modules, with the line count as a rough weight. Two of them — `tabs.py` and `docs.py` — deliberately import no Playwright and no Anthropic SDK, so their tests run in milliseconds.

Module (backend/)	Lines	What it owns
`browser.py`	904	CDP attach and auto-launch, tab discovery and selection, semantic element lookup, the six action verbs, navigation scope enforcement, screenshots and the live frame source
`session.py`	471	Per-connection orchestration: the locked preset, the sign-in gate, planning, interruption handling, teardown
`step_executor.py`	332	The run loop, the pause gate, question breaks, and mid-step plan replacement
`doc_parser.py`	255	Document + page outline -> StepPlan; re-planning; the plan sanitiser; the no-API-key fallback
`narrator.py`	201	Pre-generated narration and the mid-demo Q&A call
`overlay.py`	188	The injected cursor, spotlight and caption (browser JS)
`models.py`	171	Pydantic contracts shared by every layer
`config.py`	162	36 environment-backed settings, all with defaults

`docs.py`	155	Upload extraction (md/txt/html/PDF/docx) and storage
`tabs.py`	127	Hint -> tab scoring, and the refusal threshold
`memory.py`	88	Workflow memory: recall a plan instead of re-planning
`main.py`	339	REST surface, the WebSocket loop, health and readiness

The contracts in models.py

Everything crossing a boundary is a Pydantic model, which means the WebSocket layer, the planner and the frontend all agree by construction. `StepPlan` is also the structured-output schema handed to Claude, so the planner cannot return a shape the executor can't run.

```
ActionType   navigate | click | fill | press | wait | highlight
Action       type, target (accessible name or URL), role, value
Step         id, title, goal, doc_reference, actions[], narration
StepPlan     workflow_name, summary, steps[]        <- Claude's output schema
TabInfo      index, title, url, host, active
DemoRequest  doc_id, doc, focus, tab, workflow
QAResponse   answer, wants_plan_change, requested_focus
ClientEvent  start | question | pause | resume | skip | stop | list_tabs | reply
ServerEvent  plan | step_start | narration | screenshot | frame | step_done |
             answer | status | error | complete | tabs | prompt
```

3 · The wire protocol

Setup is REST; the demo itself is one WebSocket. The split exists because the tab picker and the document upload have to work on page load — before any session exists.

REST

Endpoint	Purpose
`GET /ping`	Cheapest possible liveness hit, for the keep-alive cron
`GET /health`	Readiness: Claude configured, Chrome reachable, auto-launch possible, and the exact command to fix Chrome by hand
`GET /tabs`	The presenter's open tabs; on failure, a hint plus the launch command
`GET /demo`	What this build runs, and whether the presenter may change it
`POST /document`	Multipart upload, 8 MB cap, returns a content-addressed id
`POST /document/text`	The same, for a pasted document
`GET /document/{id}`	What grounds the answers, so the UI can show it
`GET /samples`	Bundled example documents, one per .md in the samples directory
`GET /sample-doc`	One sample's text, by name
`GET /workflows`	Everything the agent has learned, for the resume picker

WebSocket

Inbound events are small and imperative. Outbound events are the render stream — the frontend is a fold over them and holds no other source of truth.

Outbound event	Carries

`plan`	The full StepPlan, once planning completes
`step_start / step_done`	Step id and title; drives the progress rail
`narration`	The spoken line, pre-generated at plan time
`frame`	Live JPEG at ~4 fps — image only, so captions don't flicker
`screenshot`	Post-action PNG plus the trace line and the current URL
`answer`	A reply to a mid-demo question
`prompt`	A question *for* the presenter, with buttons (the sign-in gate)
`status`	Free text plus payload: the chosen tab, the scope, question windows
`complete / error`	Terminal states

Two events are backgrounded rather than awaited on the receive loop: `question`, because Claude takes seconds and a second question must not queue behind the first; and `start`, because it blocks on the presenter answering the sign-in prompt — and that answer arrives on the same socket.

4 · The browser layer

Attaching instead of launching

`connect_over_cdp` joins the Chrome the presenter already has open. That single choice removes the credential problem — the session already exists in the profile — and puts the real product, with the real account, on the screen share. Teardown disconnects; it never closes a browser it did not open.

Chrome needs three flags, and each one is a separate afternoon if you don't know it: `--remote-debugging-port` opens the endpoint, `--user-data-dir` is mandatory since Chrome 136 (which refuses the port on the default profile), and `--enable-automation` is mandatory since Chrome 151 (which otherwise rejects the handshake). If the port is closed when a demo starts, the agent runs that command itself — but only when the CDP URL points at this machine and Chrome is installed here.

Choosing the tab

`tabs.py` scores each open tab against the presenter's hint: host equality is worth 100, a domain suffix 80, a token inside the host 40, a token in the title 12, in the path 6, and being foreground is worth 1 — a tiebreak and nothing more. Below a threshold of 10 it returns nothing and the UI shows the tab list. Opening the wrong customer's tab on a screen share is far worse than one extra question.

Finding elements

Lookup is by ARIA role plus accessible name, never CSS. Two passes: exact names first, then substrings — because short labels are common in navigation and substring matching on them is actively wrong. Within each pass the requested role is tried first, then the roles that look identical to a human, then label, placeholder and text. Only visible matches count, since apps keep duplicate markup for responsive layouts. A `fill` narrows the candidates to editable roles only and verifies the element is editable before typing.

Scope enforcement

```
def _scope_for(url):            # derived when a tab is adopted
    host = urlparse(url).hostname
    base = registrable_domain(host)
    return {host, base, *config.EXTRA_ALLOWED_HOSTS}

# checked twice: once when sanitising the plan, and again here
if not self.host_allowed(action.target):
    return f"skipped {action.target} - outside this demo's scope"
```

With no tab chosen the allowlist is empty and every navigation fails closed. The prompt asks for good behaviour; the code enforces it.

5 · The planner

Three Claude calls per run, and each one is shaped by what it costs the audience.

Call	When	Shape
Plan	Once, before execution	messages.parse with StepPlan as the output format. Input: the document, the flow in plain English, the live page outline, the allowed hosts, and whether the browser is signed in
Narrate	Once, straight after planning	One call fills every step's spoken line. Calling per step would stall every transition on a round trip
Answer	Per interruption, live	messages.parse with QAResponse, the current screenshot attached as an image block

Answering and re-planning are one call

"Can one issue live on two boards?" is a question. "Skip to the board" is a redirect. Deciding which is the same judgement as answering, so `QAResponse` carries the answer, a boolean, and the new focus together. A separate classifier call would double the pause the customer sits through.

The document is cached, not re-sent

The product document is identical across the planning, narration and every Q&A call, so it goes in a system block marked `cache_control: ephemeral`. Subsequent calls read it from cache instead of paying for the tokens again.

The sanitiser, and the fallback

Whatever the model returns passes through `_sanitise()`, which drops any navigation outside the derived scope before the plan reaches the executor. And with no API key — or a 500, or no network — `_fallback_plan()` builds steps from the document's own headings with wait-only actions. It never invents navigation. A live demo must not open with a stack trace.

6 · The executor

The smallest module that carries the most weight. It owns one `asyncio.Event` and the rule that makes interruption safe.

```
while self.current_index < len(self.plan.steps):
    if not await self._checkpoint():      # parks on the gate
        break
    step = self.plan.steps[self.current_index]

    emit(step_start); emit(narration); browser.set_caption(...)

    for action in step.actions:
```

```
        if not await self._checkpoint(): break   # gate between ACTIONS
        if self._plan_replaced:          break   # redirected mid-step
        if self._skip_requested:         break
        trace = await browser.run_action(action)
        emit(screenshot with trace)

    if self._plan_replaced:
        continue          # re-read this index; do NOT advance past it

    emit(step_done)
    self.current_index += 1        # only after the step fully completes
    await question_break(...)      # ask the room, hold, maybe re-plan
```

The three rules

The gate is an Event, not a flag	A flag forces the loop to poll — which either burns CPU or adds latency to every resume. The Event lets the coroutine park and wake the instant an answer lands.
It is checked between actions	Not just between steps. That is why an interruption freezes the browser where it stands rather than at the next step boundary.
The index advances last	`current_index` moves only after a step fully completes, so resuming continues the same step — it never rewinds and never skips.

Question breaks

After each step the executor asks the room and holds for `QUESTION_BREAK_SECONDS`; at the end it holds for a full minute. The window *restarts* whenever someone actually asks, because one question usually has a follow-up. Skip ends it early. A question asked in the closing window can still extend the demo: the outer loop re-enters if re-planning added steps, so the agent performs the new thing rather than ending on an answer nobody saw.

Pause is not instantaneous, deliberately

An action already in flight runs to completion before the loop parks — cancelling mid-action would leave a half-typed field on screen in front of the customer. Measured worst case is 1.52 s. The UI is honest about the gap: the client's request emits `pausing`, and only the executor emits the authoritative `paused`.

7 · The on-page overlay

The OS mouse pointer is not composited into screenshots or into most screen-share capture paths. Without an overlay the audience watches buttons activate by themselves, which reads as a glitch rather than as a demo. So the agent draws its own.

Cursor	An SVG arrow injected into the page, moved with a CSS transform so it glides to each target over ~0.6 s rather than teleporting
Click pulse	A ring that expands and fades on click, so activation has a cause
Spotlight	A box around the target with a large outer box-shadow that dims the rest of the page. Kept light — heavier and the product looks washed out
Caption	The current narration as a subtitle bar, which makes the browser window self-explanatory when it is shared without the app's chat panel

Two constraints it has to survive

Real products are hostile to injected DOM in two specific ways, and the overlay is built around both. **Trusted Types:** sites with that CSP throw on any HTML-string assignment, so every node is created with `createElementNS` and `textContent`, never `innerHTML`, and the whole builder is wrapped so a refusal degrades the overlay instead of killing a step. **Single-page re-renders:** the builder checks `isConnected` rather than a boolean, and restores the caption text and cursor position when it rebuilds — so an in-app navigation mid-narration is invisible.

8 · The frontend

One hook owns the socket and folds the event stream into render state. Components are presentational — no component opens a connection, and there is no second source of truth to drift.

`lib/websocket.ts`	389	The socket, the event fold, and every action the UI can take (start, ask, pause, resume, skip, stop, reply)
`components/SetupPanel.tsx`	280	Document upload/paste, the flow description, the tab picker — or, in locked mode, one card and one button
`app/page.tsx`	170	Layout, the question-window countdown, and the prompt bar
`components/ChatPanel.tsx`	110	Narration and answers as they arrive, plus the interrupt box
`lib/types.ts`	102	Mirrors models.py — kept in sync by hand, deliberately small
`components/BrowserPanel.tsx`	70	The live view: JPEG frames, then a full-quality PNG after each action
`lib/voice.ts`	49	Web Speech API narration; cancels the instant someone interrupts

The live view is two streams on purpose. Continuous JPEG frames at ~4 fps make page loads, scrolls and hovers visible — without them the panel is a slideshow of end states. The post-action PNG is full-quality and carries the trace line and URL. The frame handler deliberately touches only the image: folding the caption in there would make it flicker several times a second.

9 · Configuration

36 settings, every one with a working default, so a clean checkout runs with only `ANTHROPIC_API_KEY` set. The ones that change behaviour rather than taste:

Setting	Default	Effect
`DEMO_LOCKED`	true	Runs one fixed preset and ignores what the client sends. false restores the site-agnostic agent
`ATTACH_TO_CHROME`	true	Join the presenter's Chrome rather than launching a clean profile
`AUTO_LAUNCH_CHROME`	true	Start Chrome with the debug flags when the port is closed — local machine only
`ASK_TO_SIGN_IN`	true	Offer to wait for a sign-in before planning
`EXTRA_ALLOWED_HOSTS`	(empty)	Standing additions to the derived navigation scope
`QUESTION_BREAK_SECONDS`	12	Hold after each step. 0 runs straight through

`CLOSING_QUESTION_SECONDS`	60	The hold at the end, where questions actually arrive
`STEP_DWELL_SECONDS`	3	Pacing between steps so narration can be read
`LIVE_FPS / FRAME_QUALITY`	4 / 62	~39 KB per frame — continuous without saturating a conference-room connection
`SHOW_CURSOR / SHOW_CAPTIONS`	true	The on-page overlay

10 · How it is tested

25 hermetic tests in 2.7 seconds: no network, no Claude calls, no browser. A `FakeBrowser` records actions instead of performing them, and an autouse fixture zeroes the pacing — in a meeting the dwell and question windows are the feature, in CI they would be a minute of dead air per test.

The pause guarantee	Asserts that across a pause no action is replayed and none is dropped, and that the index lands on the same step
Mid-step re-planning	Asserts that a plan swapped in mid-step runs the new step rather than advancing past it
Question breaks	Counts the prompts, checks the closing one holds, and that a question at the end can extend the demo
Scope enforcement	The sanitiser drops out-of-scope navigation, and an empty allowlist allows nothing
Tab matching	Names the right product, defaults sensibly, and refuses below the threshold
Fill targeting	A fill never resolves to a non-editable element
Document handling	html/docx/text extraction, and content-addressed storage surviving a restart
Integration (marked)	Real browser: opens a page, derives scope, blocks an out-of-scope navigation, reads the outline, screenshots

The integration test switches attaching off and runs headless on purpose — a test must never open tabs in the developer's own Chrome.

11 · Deployment

`walkthrough-agent-skill.yaml`	Worker Agent Skill: trigger phrases, four-phase instructions (targeting, planning, execution, interruption), RBAC, audit capture, a derived domain allowlist, and denied actions. One secret: the API key
`pipeline.yaml`	Build & Test -> Register Skill -> Deploy, with StageRollback on failure. The browser smoke test reports but does not block
`rollback-policy.yaml`	Rollback triggers, session version pinning, approval gates
`.github/workflows/keep-alive.yml`	Pings /ping every two minutes so a free-tier host does not sleep between demos

Sessions pin to the skill version they started on, so a deploy cannot change the demo out from under a customer who is mid-call. The pipeline's real gate is the pause/resume suite, and the rollback policy watches for `stuck-pause` — a session that paused for a question and never resumed.

12 · Where to extend it

A new action verb	Add it to `ActionType`, handle it in `run_action`, and describe it in the planner preamble's vocabulary list. The enum is the contract — the model can only pick from it
A new document format	One branch in `docs.extract_text`. Anything unrecognised already falls back to text
A different browser	`BROWSER_CHANNEL` covers Edge and bundled Chromium for the launch path; the attach path needs any CDP endpoint
Another sample demo	Drop a .md in `backend/data/samples/`. It becomes a button, titled from its H1. A doc that states a length or step count overrides the planner's default
A different Q&A policy	`QA_PREAMBLE` in narrator.py decides what counts as a question versus a redirect — that boundary is prompt, not code

Running it

```
cd backend && python3 -m venv .venv \
  && .venv/bin/pip install -r requirements.txt \
  && .venv/bin/playwright install chromium
cp backend/.env.example backend/.env         # fill in ANTHROPIC_API_KEY
cd backend  && .venv/bin/uvicorn main:app --reload --port 8000
cd frontend && npm install && npm run dev

cd backend && .venv/bin/python -m pytest -q -m "not integration"
```

Starting a walkthrough opens a debug-enabled Chrome on a dedicated profile if one is not already running. Sign into that window once; the profile persists, so later demos skip straight past it.